EC-ElGamal Encryption

A self-contained introduction with worked examples

Companion handout to the lecture notes on Elliptic Curve Cryptography

October 20, 2026

Chapter 1

EC-ElGamal encryption, step by step

What you need to know beforehand – only four facts

1. The points of an elliptic curve E over $\mathbb{F}_p$ (plus a point at infinity $\mathcal{O}$) form a group written *additively*: we can add points $P + Q$, negate $-(x, y) = (x, -y)$, and subtract $P - Q = P + (-Q)$. The neutral element is $\mathcal{O}$.
2. G is a public base point whose multiples $G, 2G, 3G, \ldots, nG = \mathcal{O}$ form a cyclic group of (prime) order n. Computing kG is fast (double-and-add). Finding k from kG is the hard problem (ECDLP).
3. Scalar multiples commute: $k(dG) = d(kG) = (kd)G$.
4. ECDH: Alice sends $A = aG$, Bob sends $B = bG$, and both compute the same point $aB = bA = abG$.

That is all. If you remember ECDH, ElGamal will take ten minutes.

The big idea: ElGamal = ECDH + a one-time pad. In ECDH the two parties end up with a shared secret point abG and then turn it into a key. ElGamal does the same thing but with the roles arranged for *encryption*:

- Bob publishes a *long-term* public key $Q = dG$ (it plays the role of "$B = bG$" in ECDH; d is his private key).
- Alice, who wants to send a message M, picks a fresh *ephemeral* secret k and sends $C_1 = kG$ (the role of "$A = aG$").
- Now Alice and Bob *both* know the shared point $S = kQ = dC_1 = kdG$, exactly as in ECDH.
- Alice uses S as a one-time mask: she *adds* it to the message, $C_2 = M + S$. Only someone who knows S can undo this.

The ciphertext is the pair (C_1, C_2). Because Alice's secret k is fresh every time, the mask S is different every time, like a fresh one-time pad.

Dictionary: from classical ElGamal to the elliptic-curve version. If you know ElGamal in $\mathbb{Z}_p^\times$ (as in Paar's book), the elliptic-curve version is the same scheme with the group written additively: *multiplication becomes addition* and *exponentiation becomes scalar multiplication.*

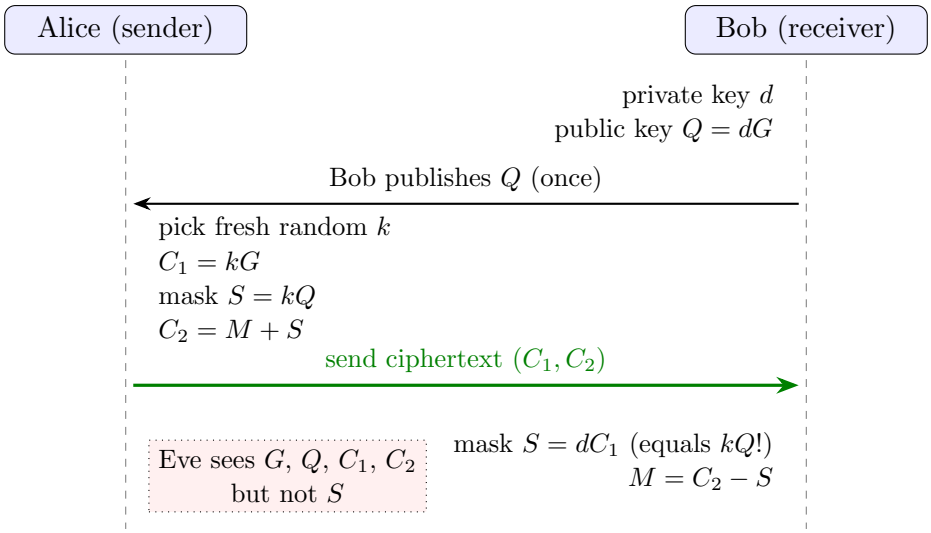


Figure 1.1: EC-ElGamal as ECDH plus masking. The point $C_1 = kG$ is Alice's ECDH public value; the shared point $S = kQ = dC_1$ is the ECDH secret; it is used to hide M by adding it.

	Classical ElGamal in $\mathbb{Z}_p^\times$	EC-ElGamal on $E(\mathbb{F}_p)$
public base	g	G
private key	d	d
public key	$\beta = g^d$	$Q = dG$
message	integer $m \in \mathbb{Z}_p^\times$	a *point* $M \in \langle G \rangle$
ephemeral secret	k	k
first part	$c_1 = g^k$	$C_1 = kG$
mask	β^k	kQ
second part	$c_2 = m \cdot \beta^k$	$C_2 = M + kQ$
decryption	$m = c_2 \cdot (c_1^d)^{-1}$	$M = C_2 - dC_1$

The scheme

Definition 1.1 (EC-ElGamal)**.** Public parameters: curve E over $\mathbb{F}_p$, point G of prime order n.

1. **Key generation (Bob).** Choose $d \xleftarrow{\$} \{1, \dots, n-1\}$ uniformly. Public key $Q = dG$; private key d.
2. **Encryption (Alice)** of a message point $M \in \langle G \rangle$: choose a fresh random $k \xleftarrow{\$} \{1, \dots, n-1\}$ and output $\big(C_1, C_2\big) = \big(kG,\ M + kQ\big)$.
3. **Decryption (Bob)**: compute $M = C_2 - dC_1$.

Theorem 1.2 (Correctness)**.** *Decryption returns M.*

Proof. $dC_1 = d(kG) = k(dG) = kQ$, hence $C_2 - dC_1 = (M + kQ) - kQ = M$. □

Cost. Encryption needs two scalar multiplications (kG and kQ) and one point addition; decryption needs one scalar multiplication and one subtraction. The ciphertext consists of *two* points, i.e. it is about twice as long as the message point.

A problem you must solve first: messages are not points

The scheme encrypts a *point* M, but we want to encrypt numbers, text, keys. The classical (Koblitz) fix is to embed a small integer m into a point:

Koblitz embedding with parameter κ

To encode $m \in \{0, 1, \ldots, \lfloor p/\kappa \rfloor - 1\}$: try $x = \kappa m,\ \kappa m + 1,\ \ldots,\ \kappa m + \kappa - 1$ and take the first x for which $x^3 + ax + b$ is a square modulo p (test with Euler's criterion); let y be a square root (choose the smaller one). Then $M_m = (x, y)$. To decode, output $m = \lfloor x/\kappa \rfloor$ (only the x-coordinate is needed). Each trial succeeds with probability about $\frac{1}{2}$, so encoding fails with probability about $2^{-\kappa}$; real systems take $\kappa \approx 30$–60.

We will work on a curve small enough to do everything by hand.

The toy curve. $E:\ y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$, with base point $G = (5, 1)$ of prime order $n = 19$. All multiples of G (you will use this table all the time):

k	1	2	3	4	5	6	7	8	9
kG	$(5,1)$	$(6,3)$	$(10,6)$	$(3,1)$	$(9,16)$	$(16,13)$	$(0,6)$	$(13,7)$	$(7,6)$
k	10	11	12	13	14	15	16	17	18
kG	$(7,11)$	$(13,10)$	$(0,11)$	$(16,4)$	$(9,1)$	$(3,16)$	$(10,11)$	$(6,14)$	$(5,16)$

Remember $-kG = (19 - k)G$, i.e. negation flips $y \mapsto -y$; for instance $-(6, 3) = (6, 14)$. And $19G = \mathcal{O}$.

Embedding table for $\kappa = 3$, so $m \in \{0, \ldots, 4\}$ (the squares modulo 17 are $1, 2, 4, 8, 9, 13, 15, 16$; one checks that $f(x) = x^3 + 2x + 2 \bmod 17$ is a square exactly for $x = 0, 3, 5, 6, 7, 9, 10, 13, 16$):

m	0	1	2	3	4
x tried	0 ✓	3 ✓	6 ✓	9 ✓	12 (no), 13 ✓
M_m	$(0,6)$	$(3,1)$	$(6,3)$	$(9,1)$	$(13,7)$
$=$	$7G$	$4G$	$2G$	$14G$	$8G$

Worked example 1: encrypt and decrypt

Example 1.3 (Bob's key pair)**.** Bob chooses the private key $d = 7$ and computes $Q = 7G = (0, 6)$ (table). He publishes Q.

Example 1.4 (Alice encrypts the message $m = 4$)**.** Alice wants to send $m = 4$. She embeds it: $M = (13, 7) = 8G$. She chooses the fresh random nonce $k = 3$ and computes

$$\begin{aligned} C_1 &= kG = 3G = (10, 6), \\ S &= kQ = 3 \cdot (0, 6) = 3 \cdot 7G = 21G = 2G = (6, 3) \qquad \text{(the mask)}, \\ C_2 &= M + S = 8G + 2G = 10G = (7, 11). \end{aligned}$$

She sends $(C_1, C_2) = \big((10, 6), (7, 11)\big)$.

Example 1.5 (Bob decrypts)**.** Bob computes the mask from C_1 with his private key: $S' = dC_1 = 7 \cdot (10, 6) = 7 \cdot 3G = 21G = 2G = (6, 3)$. This is the *same* point S that Alice used. He subtracts it:

$$M = C_2 - S' = (7, 11) + \big(-(6, 3)\big) = (7, 11) + (6, 14).$$

The slope is $\lambda = \frac{14-11}{6-7} = \frac{3}{-1} \equiv 14$, so $x_3 = 14^2 - 7 - 6 = 183 \equiv 13$ and $y_3 = 14(7 - 13) - 11 = -95 \equiv 7$, giving $M = (13, 7)$. Decoding: $m = \lfloor 13/3 \rfloor = 4$. ✓

What does Eve see? Only $G = (5, 1)$, $Q = (0, 6)$, $C_1 = (10, 6)$, $C_2 = (7, 11)$. To recover M she needs the mask $S = kQ = (6, 3)$. Computing it needs k (from $C_1 = kG$, an ECDLP) or d (from $Q = dG$, an ECDLP). On this toy curve she simply reads $k = 3$ off the table; on a 256-bit curve that is infeasible. Computing S from (G, Q, C_1) without either secret is exactly the Diffie–Hellman problem CDH.

Example 1.6 (Same message, new ciphertext)**.** Alice encrypts $m = 4$ again with a new nonce $k = 11$: $C_1 = 11G = (13, 10)$, $S = 11Q = 77G = G = (5, 1)$ (as $77 = 4 \cdot 19 + 1$), $C_2 = 8G + G = 9G = (7, 6)$. The ciphertext $\big((13, 10), (7, 6)\big)$ looks completely different, yet Bob decrypts it: $dC_1 = 7 \cdot 11G = 77G = G$ and $9G - G = 8G = (13, 7)$. ✓This randomisation is essential: if encryption were deterministic, anyone could encrypt guesses and compare.

Why is it secure?

Theorem 1.7 (Semantic security)**.** *If the Decisional Diffie–Hellman problem is hard in $\langle G \rangle$, EC-ElGamal is IND-CPA secure: an adversary who sees the public key and a ciphertext learns nothing about which of two chosen messages was encrypted.*

Idea. The adversary sees $Q = dG$, $C_1 = kG$ and $C_2 = M + S$. If $S = kdG$ were replaced by a uniformly random point R, then $C_2 = M + R$ would be uniformly distributed whatever M is, exactly as with a one-time pad, and would carry no information about M. DDH hardness says that the genuine $S = kdG$ cannot be distinguished from such a random R given (G, dG, kG). So the real ciphertext is indistinguishable from one that hides M perfectly. □

Intuition. The point S is a "fresh one-time pad" which only Alice (who knows k) and Bob (who knows d) can compute. This is the same fact that makes ECDH secure.

Three pitfalls (and what they teach)

Pitfall 1: never reuse the nonce k. If two messages are encrypted with the same k, the same mask S is added twice, like reusing a one-time pad.

Example 1.8 (Nonce reuse)**.** Alice encrypts $M_1 = (13, 7) = 8G$ and $M_2 = (6, 3) = 2G$, both with $k = 3$. The mask is $S = 2G$ in both cases, so C_1 is the same and $C_2 = 8G + 2G = 10G = (7, 11)$, $C_2' = 2G + 2G = 4G = (3, 1)$. Eve notices the repeated $C_1 = (10, 6)$ and computes $C_2 - C_2' = (7, 11) - (3, 1) = 6G = (16, 13) = M_1 - M_2$. If she also knows (or guesses) one plaintext, say M_1, she gets the mask $S = C_2 - M_1 = (6, 3)$ and then $M_2 = C_2' - S = (3, 1) - (6, 3) = 2G = (6, 3)$. All without the private key.

The nonce must be uniformly random and fresh for *every* encryption (a bad random-number generator is as bad as reuse).

Pitfall 2: ciphertexts can be modified (malleability).

Example 1.9 (An attacker edits the plaintext)**.** Take Alice's ciphertext $\big((10, 6), (7, 11)\big)$ of $m = 4$ from Theorem 1.4. Mallory replaces C_2 by $C_2 + G = (7, 11) + (5, 1) = 11G = (13, 10)$ and forwards $\big((10, 6), (13, 10)\big)$. Bob decrypts $11G - 2G = 9G = (7, 6)$ and decodes $m = \lfloor 7/3 \rfloor = 2$. The message changed from 4 to 2, and Bob notices nothing, although Mallory never learned the key.

In general adding a point T to C_2 changes the plaintext to $M + T$. This is why EC-ElGamal alone is not secure against *active* attackers (it is not IND-CCA2). Practical systems use ECIES (Lecture 4), which authenticates the ciphertext.

Pitfall 3 (a feature in disguise): the scheme is additively homomorphic.

Example 1.10 (Adding encrypted messages)**.** Encrypt $M_1 = (13, 7) = 8G$ with $k = 3$: $(C_1, C_2) = \big((10, 6), (7, 11)\big)$. Encrypt $M_2 = (3, 1) = 4G$ (this is $m = 1$) with $k = 11$: $(C_1', C_2') = \big((13, 10), (9, 16)\big)$. Add the ciphertexts component-wise:

$$\big(C_1 + C_1',\ C_2 + C_2'\big) = \big((10, 6) + (13, 10),\ (7, 11) + (9, 16)\big) = \big((9, 1),\ (3, 16)\big).$$

Bob decrypts: $d(9, 1) = 7 \cdot 14G = 98G = 3G = (10, 6)$, and $(3, 16) - (10, 6) = 15G - 3G = 12G = (0, 11)$. And indeed $M_1 + M_2 = 8G + 4G = 12G = (0, 11)$. ✓
So one can add plaintexts *without decrypting*. This is used in electronic voting: encode a vote $v \in \{0, 1\}$ as the point vG ("exponential ElGamal"), add all encrypted ballots, decrypt once to get $(\sum v)G$ and recover the tally $\sum v$ by trying $0G, 1G, 2G, \ldots$ (cheap for small tallies).

Practical remarks.

- Before using a received ciphertext, check that C_1, C_2 are valid points of the curve in the correct subgroup (otherwise invalid-curve attacks, Lecture 4); reject $C_1 = \mathcal{O}$.
- The message M must lie in the subgroup $\langle G \rangle$; on prime-order curves that is every point.
- The embedded message is short (about 250 bits on a 256-bit curve). Real systems encrypt long data with a symmetric cipher and use ElGamal only to transport the key: this is **ECIES**: $C_1 = kG$, key $= \mathrm{KDF}(x(kQ))$, then authenticated encryption of the data. ECIES avoids both the embedding problem and malleability.

Python implementation (toy curve)

The following code reproduces every number in this section (it uses the `Curve` class of the appendix with $E(\mathbb{F}_{17})$). It is for learning only.

```
import secrets
from ecc_toy import Curve # class Curve from the appendix

E = Curve(17, 2, 2, G=(5, 1), n=19) # y^2 = x^3 + 2x + 2 over F_17, order of G = 19
KAPPA = 3

def keygen():
    d = secrets.randbelow(E.n - 1) + 1 # private key
    return d, E.mul(d, E.G) # (d, Q = dG)

def embed(m, kappa=KAPPA): # integer -> curve point (Koblitz)
    for j in range(kappa):
        x = kappa * m + j
        rhs = (x**3 + E.a * x + E.b) % E.p
        ys = [y for y in range(E.p) if y * y % E.p == rhs]
        if ys: return (x, min(ys))
    raise ValueError("embedding failed")

def unembed(P, kappa=KAPPA): # curve point -> integer
    return P[0] // kappa

```

```python
def encrypt(Q, M, k=None):
    k = k or secrets.randbelow(E.n - 1) + 1 # FRESH random nonce
    return E.mul(k, E.G), E.add(M, E.mul(k, Q)) # (C1, C2) = (kG, M + kQ)

def decrypt(d, C):
    C1, C2 = C
    return E.add(C2, E.neg(E.mul(d, C1))) # M = C2 - d*C1

d, Q = 7, E.mul(7, E.G)
M = embed(4) # (13, 7)
C = encrypt(Q, M, k=3) # ((10, 6), (7, 11))
print(C, decrypt(d, C), unembed(decrypt(d, C))) # ... (13, 7) 4
```

One-page summary

- **Keys:** private d; public $Q = dG$.
- **Encrypt** M**:** pick random k; send $\big(kG,\ M + kQ\big)$.
- **Decrypt:** $M = C_2 - dC_1$, because $dC_1 = kQ$ (the ECDH secret).
- **Security:** kQ is a fresh one-time pad; secure if DDH is hard.
- **Must do:** fresh random k every time; embed messages as points; validate points.
- **Limits:** malleable (edit C_2 edits M), ciphertext twice the size, short messages $\Rightarrow$ use ECIES in practice.
- **Bonus:** additively homomorphic $\Rightarrow$ e-voting, private sums.

Exercises (with answers)

Exercise 1.1. With Bob's key $d = 7$, $Q = (0, 6)$ encrypt $M = (9, 1)$ (this is $m = 3$) using $k = 2$, then decrypt. *Answer:* $C_1 = 2G = (6, 3)$, $S = 2Q = 14G = (9, 1)$, $C_2 = 14G + 14G = 28G = 9G = (7, 6)$; ciphertext $\big((6, 3), (7, 6)\big)$. Decryption: $dC_1 = 14G$ and $9G - 14G = -5G = 14G = (9, 1)$.

Exercise 1.2. Decrypt $\big((13, 10), (0, 11)\big)$ with $d = 7$ and decode the message. *Answer:* $C_1 = 11G$, $dC_1 = 77G = G$, $M = 12G - G = 11G = (13, 10)$, so $m = \lfloor 13/3 \rfloor = 4$ (decoding only uses the x-coordinate).

Exercise 1.3. What happens if Alice chooses $k = 19$ (or $k = 0$)? *Answer:* $C_1 = \mathcal{O}$ and $S = \mathcal{O}$, so $C_2 = M$: the plaintext is sent in the clear. This is why k is drawn from $\{1, \ldots, n - 1\}$.

Exercise 1.4. Two ciphertexts $\big((10, 6), (7, 11)\big)$ and $\big((10, 6), (3, 1)\big)$ are intercepted, and Eve knows the first plaintext is $(13, 7)$. Recover the second. *Answer:* the same C_1 means the same k; $S = (7, 11) - (13, 7) = (6, 3)$; $M_2 = (3, 1) - (6, 3) = (6, 3)$, i.e. $m = 2$.

Exercise 1.5. In an election with encrypted votes $v_i \in \{0, 1\}$ encoded as $v_i G$, explain how to obtain the tally without decrypting individual votes, and why it only works for small tallies. *Answer:* add all ciphertexts (component-wise), decrypt the sum once to get TG, $T = \sum v_i$; find T by testing $0G, 1G, \ldots$ (a discrete logarithm of a small number: cheap only if T is at most about 2^{40}).

Exercise 1.6. Show that $(C_1, 2C_2)$ does *not* decrypt to $2M$, but $(2C_1, 2C_2)$ does. *Answer:* decrypting $(C_1, 2C_2)$ gives $2C_2 - dC_1 = 2M + kQ \neq 2M$, whereas $(2C_1, 2C_2)$ gives $2C_2 - 2dC_1 = 2M + 2kQ - 2kQ = 2M$.